Q/ROUTE, explained simply

How the mission editor, pathfinding and quantum optimization work together.

The mission

One drone starts at depot **00**, visits delivery checkpoints **01, 02 and 03**, and returns to the depot. It must avoid obstacles and electronic warfare (EW) zones while keeping travel cost low.

Part of the system	Its job
Classical Dijkstra search	Find safe cell-by-cell paths between checkpoints.
QAOA	Choose the order in which the four locations are visited.
Exact classical solver	Check all six depot-first orders to verify the optimum.
Frontend playback	Move one drone along the actual computed grid paths.

1. Edit and validate the environment

Drag checkpoints or EW zones, resize zones, and draw or erase obstacles. Each edit clears the previous quantum route and increments the environment revision. Old responses are ignored. Cells touching an EW circle are conservatively blocked.

If a checkpoint is blocked or disconnected, the app reports **No feasible route** and skips QAOA. This is a proven connectivity problem, not a quantum sampling failure.

2. Find paths and build the cost matrix

Dijkstra searches the six pairs of locations. It records each safe shortest path, its length, explored cells and frontier discoveries. The blue and sand animation replays those real search records.

These path lengths form a matrix: **W[i][j]** is the safe travel cost from location i to j. QAOA and exact classical verification both use this same matrix. QAOA does not search the grid cells.

3. Encode the visiting order

A binary variable **x[i,p]** means "node i is assigned to tour position p." Four nodes times four positions gives **16 binary variables and 16 qubits**. Every node must appear once, and every position must contain one node.

Qiskit converts travel costs and assignment-violation penalties into a **QUBO**, then an **Ising Hamiltonian**: an energy objective for the circuit. Lower energy is desirable. The closing journey back to the depot is included.

The quantum controls

Layers build the circuit. Evaluations tune it. Shots sample its output.

4. QAOA layers

QAOA means Quantum Approximate Optimization Algorithm. Each layer applies a cost operation and a mixer operation. Together they change the probabilities of candidate assignments through quantum interference.

Each layer has two adjustable angles, gamma and beta. **Two layers therefore have four angles.** More layers usually increase circuit depth and tuning work, but do not guarantee a better route. Layer count is not the same as gate-level circuit depth.

5. COBYLA evaluations

COBYLA is a classical optimizer from SciPy. It proposes circuit angles, asks Qiskit to evaluate expected energy, and uses that result to propose another set of angles. It does not need derivatives.

At **40 max evaluations**, it gets up to 40 objective evaluations and may stop earlier. It tunes the average energy of the circuit distribution, not one decoded route. The history chart shows QUBO expectation, including assignment penalties.

6. Sampling shots

After optimization, **4,096 shots** means drawing 4,096 measurement outcomes from the tuned circuit. Each outcome is a 16-bit assignment. Invalid assignments are rejected; the app selects the lowest-cost feasible sampled tour.

Your app uses exact simulated expectations while tuning and sampling afterward. It does **not** take 4,096 measurement shots at each COBYLA evaluation.

Control	What changing it affects
COBYLA evaluations	Parameter-search budget and solver runtime.
QAOA layers	Circuit structure and number of angles.
Shots	Number of final sampled outcomes.
Animation speed	Playback only; it does not change the answer.

7. Decode, compare and animate

For an order such as **00 - 01 - 03 - 02 - 00**, the app joins the corresponding Dijkstra paths. The drone follows those cells and pauses at targets. Exact enumeration checks whether its total cost equals the optimum.

Converged means COBYLA met its stopping criteria; it does not prove global optimality. **No feasible QAOA sample** means the map is reachable but sampling found no valid assignment. A frequent bitstring can still be invalid.

Where Qiskit fits

Qiskit defines the assignment problem, converts QUBO to Ising, builds the QAOA circuit, evaluates expected energy and samples outcomes. It runs locally on your CPU: **no physical IBM quantum processor is used, and no quantum speedup is claimed.**

References: SciPy COBYLA | IBM StatevectorEstimator | IBM QAOA ansatz